

**LAPORAN PRAKTIKUM ALGORITMA
DAN PEMROGRAMAN 1**

**MODUL 18
“MESIN ABSTRAK”**



Disusun Oleh :

Nama : Bagus Irsyad Kamal
NIM : 109082500215
Kelas : S1-IF-13-02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng.

**PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026**

A. Tugas Mandiri (soal tugas, berdasarkan file tugas yang diberikan)

Tugas 18.3.1

```
package main

import (
    "fmt"
    "math/rand"
)

type Domino struct {
    suit  [2]int
    nilai int
    isBalak bool
}

type Dominoes struct {
    kartu    [28]Domino
    jumlahTersisa int
}

func buatDomino(s1, s2 int) Domino {
    return Domino{
        suit:  [2]int{s1, s2},
        nilai: s1 + s2,
        isBalak: s1 == s2,
    }
}

func buatDominoes() Dominoes {
    var d Dominoes
    idx := 0
    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
            d.kartu[idx] = buatDomino(i, j)
            idx++
        }
    }
    d.jumlahTersisa = 28
    return d
}

func kocokKartu(d *Dominoes) {
    for i := 27; i > 0; i-- {
        j := rand.Intn(i + 1)
        d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i]
    }
}
```

```

func ambilKartu(d *Dominoes) Domino {
    if d.jumlahTersisa == 0 {
        fmt.Println("Tidak ada kartu tersisa!")
        return Domino{}
    }
    kartu := d.kartu[d.jumlahTersisa-1]
    d.jumlahTersisa--
    return kartu
}

func gambarKartu(d Domino, suit int) int {
    if suit == 0 {
        return d.suit[0]
    }
    return d.suit[1]
}

func nilaiKartu(d Domino) int {
    return d.nilai
}

func cetakKartu(d Domino) {
    balak := "bukan balak"
    if d.isBalak {
        balak = "balak"
    }
    fmt.Printf("[%d|%d] nilai=%d (%s)\n", d.suit[0], d.suit[1], d.nilai, balak)
}

func main() {
    set := buatDominoes()
    fmt.Println("=== Set Domino Sebelum Dikocok ===")
    for i := 0; i < 5; i++ {
        cetakKartu(set.kartu[i])
    }

    kocokKartu(&set)
    fmt.Println("\n=== Set Domino Setelah Dikocok (5 kartu pertama) ===")
    for i := 0; i < 5; i++ {
        cetakKartu(set.kartu[i])
    }

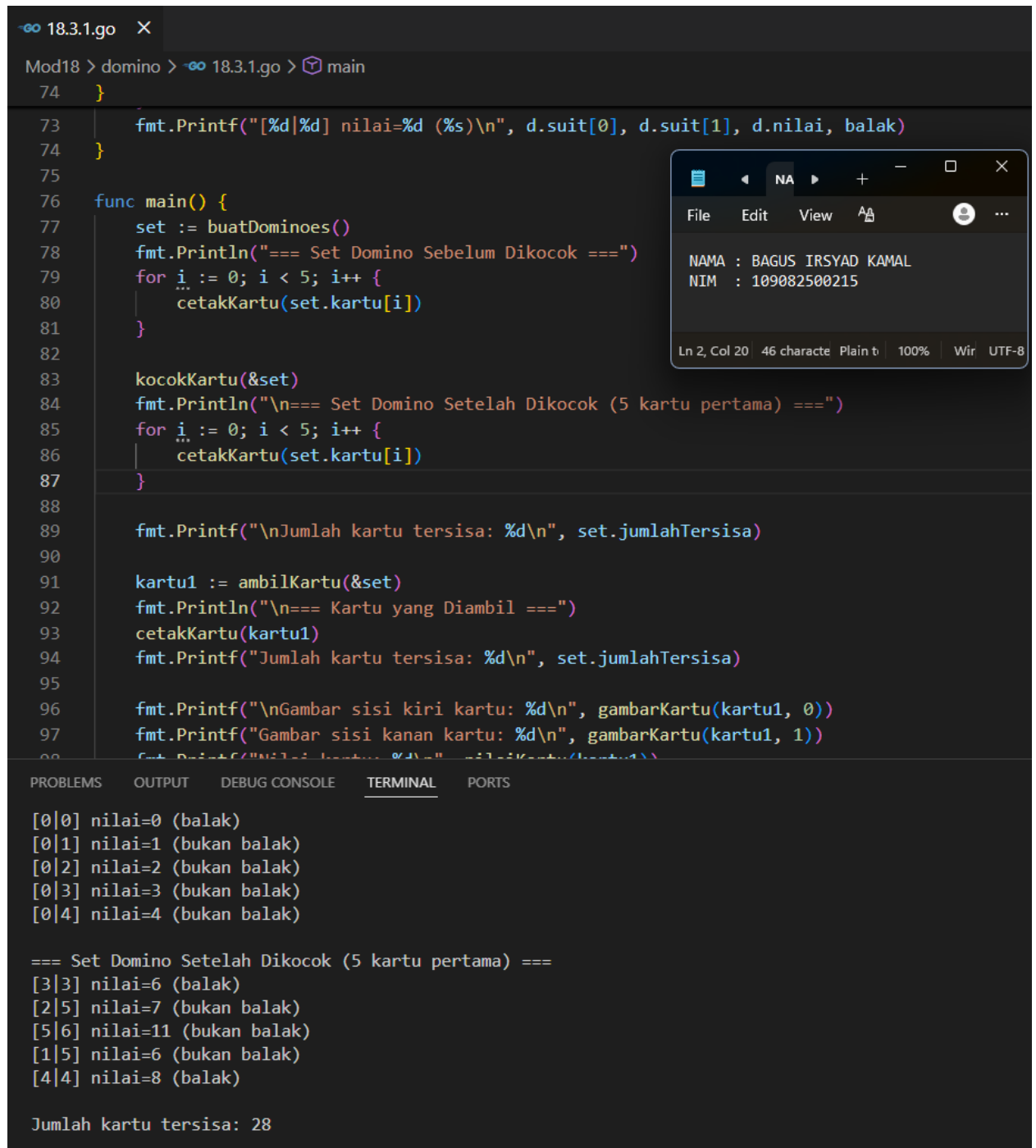
    fmt.Printf("\nJumlah kartu tersisa: %d\n", set.jumlahTersisa)

    kartu1 := ambilKartu(&set)
    fmt.Println("\n=== Kartu yang Diambil ===")
    cetakKartu(kartu1)
    fmt.Printf("Jumlah kartu tersisa: %d\n", set.jumlahTersisa)
}

```

```
    fmt.Printf("\nGambar sisi kiri kartu: %d\n", gambarKartu(kartu1, 0))  
    fmt.Printf("Gambar sisi kanan kartu: %d\n", gambarKartu(kartu1, 1))  
    fmt.Printf("Nilai kartu: %d\n", nilaiKartu(kartu1))  
}
```

Screenshots Output



The screenshot displays a Go IDE with a dark theme. The editor window shows a Go program for a domino game. The code includes functions for creating dominoes, printing them, drawing cards, and calculating the remaining cards. A small window titled 'NA' is overlaid on the code, displaying the user's name and NIM. The terminal window at the bottom shows the execution output, including the initial domino values, the set of dominoes after drawing, the cards drawn, and the final count of remaining cards.

```
18.3.1.go X
Mod18 > domino > 18.3.1.go > main
74 }
73 fmt.Printf("[%d|%d] nilai=%d (%s)\n", d.suit[0], d.suit[1], d.nilai, balak)
74 }
75
76 func main() {
77     set := buatDominoes()
78     fmt.Println("=== Set Domino Sebelum Dikocok ===")
79     for i := 0; i < 5; i++ {
80         cetakKartu(set.kartu[i])
81     }
82
83     kocokKartu(&set)
84     fmt.Println("\n=== Set Domino Setelah Dikocok (5 kartu pertama) ===")
85     for i := 0; i < 5; i++ {
86         cetakKartu(set.kartu[i])
87     }
88
89     fmt.Printf("\nJumlah kartu tersisa: %d\n", set.jumlahTersisa)
90
91     kartu1 := ambilKartu(&set)
92     fmt.Println("\n=== Kartu yang Diambil ===")
93     cetakKartu(kartu1)
94     fmt.Printf("Jumlah kartu tersisa: %d\n", set.jumlahTersisa)
95
96     fmt.Printf("\nGambar sisi kiri kartu: %d\n", gambarKartu(kartu1, 0))
97     fmt.Printf("Gambar sisi kanan kartu: %d\n", gambarKartu(kartu1, 1))
98     fmt.Printf("Nilai kartu: %d\n", nilaiKartu(kartu1))
99 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[0|0] nilai=0 (balak)
[0|1] nilai=1 (bukan balak)
[0|2] nilai=2 (bukan balak)
[0|3] nilai=3 (bukan balak)
[0|4] nilai=4 (bukan balak)

=== Set Domino Setelah Dikocok (5 kartu pertama) ===
[3|3] nilai=6 (balak)
[2|5] nilai=7 (bukan balak)
[5|6] nilai=11 (bukan balak)
[1|5] nilai=6 (bukan balak)
[4|4] nilai=8 (balak)

Jumlah kartu tersisa: 28
```

Deskripsi:

Program ini merupakan implementasi soal 18.3.1 Modul 18. Tujuannya mensimulasikan operasi dasar mesin domino sebagai subprogram, mulai dari membuat set kartu, mengocok, mengambil kartu, hingga membaca informasi kartu secara terpisah.

Tipe data yang digunakan:

Domino menyimpan informasi sebuah kartu domino, yaitu gambar kedua sisi kartu dalam array dua elemen (suit [2]int), nilai kartu (nilai int berisi jumlah kedua sisi), dan boolean isBalak yang menyatakan apakah kartu tersebut balak atau bukan. Dominoes menyimpan satu set lengkap kartu domino, yaitu array 28 kartu (kartu [28]Domino) dan jumlah kartu yang tersisa (jumlahTersisa int).

Fungsi-fungsi yang digunakan:

buatDomino menerima dua nilai integer sisi kartu, lalu mengembalikan struct Domino lengkap dengan nilai dan status balaknya. buatDominoes membuat satu set lengkap 28 kartu domino dengan dua perulangan bersarang dari 0 hingga 6, menghasilkan semua kombinasi unik kartu domino, lalu mengembalikan struct Dominoes dengan jumlahTersisa diset 28. kocokKartu menerima pointer Dominoes lalu mengacak urutan kartu menggunakan algoritma Fisher-Yates, menukar setiap kartu dari belakang ke depan dengan kartu di posisi acak. ambilKartu menerima pointer Dominoes, mengambil kartu teratas dari tumpukan, mengurangi jumlahTersisa sebanyak satu, lalu mengembalikan kartu tersebut. gambarKartu menerima sebuah Domino dan indeks sisi (suit int), lalu mengembalikan nilai gambar sisi kiri jika suit bernilai 0, atau sisi kanan jika suit bernilai 1. nilaiKartu menerima sebuah Domino lalu mengembalikan total nilai kartu yaitu jumlah kedua sisinya.

main membuat satu set domino lengkap, menampilkan 5 kartu pertama sebelum dikocok, lalu mengocok seluruh kartu dan menampilkan 5 kartu pertama setelah dikocok. Kemudian program mengambil satu kartu, menampilkan informasi lengkapnya termasuk hasil gambarKartu untuk kedua sisi dan nilaiKartu, serta menampilkan jumlah kartu yang tersisa.

Tugas 18.3.2

```
package main

import (
    "fmt"
    "math/rand"
)

type Domino struct {
    suit  [2]int
    nilai int
    isBalak bool
}

type Dominoes struct {
    kartu    [28]Domino
    jumlahTersisa int
}

func buatDomino(s1, s2 int) Domino {
    return Domino{
        suit:  [2]int{s1, s2},
        nilai: s1 + s2,
        isBalak: s1 == s2,
    }
}

func buatDominoes() Dominoes {
    var d Dominoes
    idx := 0
    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
            d.kartu[idx] = buatDomino(i, j)
            idx++
        }
    }
    d.jumlahTersisa = 28
    return d
}

func kocokKartu(d *Dominoes) {
    for i := 27; i > 0; i-- {
        j := rand.Intn(i + 1)
        d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i]
    }
}

func ambilKartu(d *Dominoes) Domino {
    if d.jumlahTersisa == 0 {
        fmt.Println("Tidak ada kartu tersisa!")
    }
}
```

```

    return Domino{}
}
kartu := d.kartu[d.jumlahTersisa-1]
d.jumlahTersisa--
return kartu
}

func gambarKartu(d Domino, suit int) int {
    if suit == 0 {
        return d.suit[0]
    }
    return d.suit[1]
}

func nilaiKartu(d Domino) int {
    return d.nilai
}

func cetakKartu(d Domino) {
    balak := "bukan balak"
    if d.isBalak {
        balak = "balak"
    }
    fmt.Printf("[%d|%d] nilai=%d (%s)\n", d.suit[0], d.suit[1], d.nilai, balak)
}

func galiKartu(d *Dominoes, target Domino) Domino {
    var hasil Domino
    for d.jumlahTersisa > 0 {
        hasil = ambilKartu(d)
        if hasil.suit[0] == target.suit[0] || hasil.suit[0] == target.suit[1] ||
            hasil.suit[1] == target.suit[0] || hasil.suit[1] == target.suit[1] {
            return hasil
        }
    }
    return hasil
}

func sepasangKartu(a, b Domino) bool {
    return nilaiKartu(a)+nilaiKartu(b) == 12
}

func main() {
    set := buatDominoes()
    kocokKartu(&set)

    fmt.Println("=== Set Domino Setelah Dikocok (5 kartu pertama) ===")
    for i := 0; i < 5; i++ {
        cetakKartu(set.kartu[set.jumlahTersisa-1-i])
    }
}

```

```

    fmt.Println("\n=== galiKartu ===")
    target := ambilKartu(&set)
    fmt.Print("Kartu target: ")
    cetakKartu(target)
    fmt.Printf("Jumlah kartu tersisa setelah ambil target: %d\n", set.jumlahTersisa)

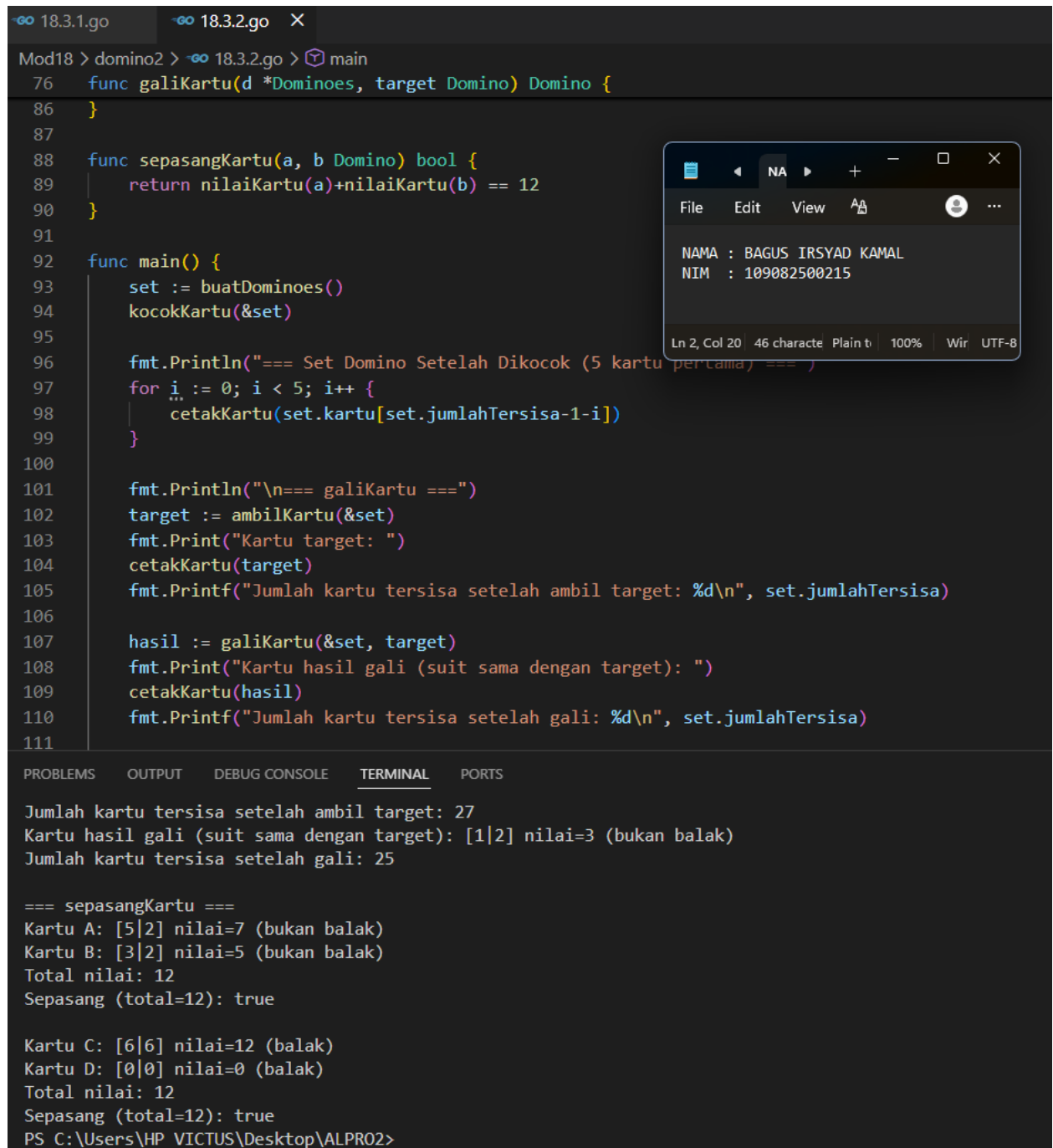
    hasil := galiKartu(&set, target)
    fmt.Print("Kartu hasil gali (suit sama dengan target): ")
    cetakKartu(hasil)
    fmt.Printf("Jumlah kartu tersisa setelah gali: %d\n", set.jumlahTersisa)

    fmt.Println("\n=== sepasangKartu ===")
    kartuA := buatDomino(5, 2)
    kartuB := buatDomino(3, 2)
    fmt.Print("Kartu A: ")
    cetakKartu(kartuA)
    fmt.Print("Kartu B: ")
    cetakKartu(kartuB)
    fmt.Printf("Total nilai: %d\n", nilaiKartu(kartuA)+nilaiKartu(kartuB))
    fmt.Printf("Sepasang (total=12): %v\n", sepasangKartu(kartuA, kartuB))

    kartuC := buatDomino(6, 6)
    kartuD := buatDomino(0, 0)
    fmt.Print("\nKartu C: ")
    cetakKartu(kartuC)
    fmt.Print("Kartu D: ")
    cetakKartu(kartuD)
    fmt.Printf("Total nilai: %d\n", nilaiKartu(kartuC)+nilaiKartu(kartuD))
    fmt.Printf("Sepasang (total=12): %v\n", sepasangKartu(kartuC, kartuD))
}

```

Screenshots Output



The screenshot displays a Go IDE with two tabs: 18.3.1.go and 18.3.2.go. The active tab, 18.3.2.go, contains the following Go code:

```
Mod18 > domino2 > 18.3.2.go > main
76 func galiKartu(d *Dominoes, target Domino) Domino {
86 }
87
88 func sepasangKartu(a, b Domino) bool {
89     return nilaiKartu(a)+nilaiKartu(b) == 12
90 }
91
92 func main() {
93     set := buatDominoes()
94     kocokKartu(&set)
95
96     fmt.Println("=== Set Domino Setelah Dikocok (5 kartu pertama) === ")
97     for i := 0; i < 5; i++ {
98         cetakKartu(set.kartu[set.jumlahTersisa-1-i])
99     }
100
101     fmt.Println("\n=== galiKartu ===")
102     target := ambilKartu(&set)
103     fmt.Print("Kartu target: ")
104     cetakKartu(target)
105     fmt.Printf("Jumlah kartu tersisa setelah ambil target: %d\n", set.jumlahTersisa)
106
107     hasil := galiKartu(&set, target)
108     fmt.Print("Kartu hasil gali (suit sama dengan target): ")
109     cetakKartu(hasil)
110     fmt.Printf("Jumlah kartu tersisa setelah gali: %d\n", set.jumlahTersisa)
111 }
```

A small window titled "NA" is overlaid on the code, displaying the following text:

```
NAMA : BAGUS IRSYAD KAMAL
NIM : 109082500215
```

The terminal output at the bottom shows the execution results:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Jumlah kartu tersisa setelah ambil target: 27
Kartu hasil gali (suit sama dengan target): [1|2] nilai=3 (bukan balak)
Jumlah kartu tersisa setelah gali: 25

=== sepasangKartu ===
Kartu A: [5|2] nilai=7 (bukan balak)
Kartu B: [3|2] nilai=5 (bukan balak)
Total nilai: 12
Sepasang (total=12): true

Kartu C: [6|6] nilai=12 (balak)
Kartu D: [0|0] nilai=0 (balak)
Total nilai: 12
Sepasang (total=12): true
PS C:\Users\HP VICTUS\Desktop\ALPRO2>
```

Deskripsi:

Program ini merupakan pengembangan dari soal 18.3.1. Tujuannya merealisasikan aksi-aksi lanjutan mesin domino menggunakan operasi-operasi dasar yang sudah dibuat sebelumnya, yaitu menggali kartu berdasarkan kesamaan suit dan mengecek apakah dua kartu membentuk pasangan dengan total nilai 12.

Fungsi-fungsi yang digunakan:

`galiKartu` menerima pointer `Dominoes` dan sebuah `Domino` sebagai kartu target, lalu mengambil kartu dari tumpukan satu per satu menggunakan `ambilKartu` hingga ditemukan kartu yang memiliki setidaknya satu sisi (suit) yang nilainya sama dengan salah satu sisi kartu target. Begitu kartu yang cocok ditemukan, fungsi langsung mengembalikannya. Jika seluruh tumpukan habis tanpa menemukan kartu yang cocok, kartu terakhir yang diambil dikembalikan sebagai hasil.

`sepasangKartu` menerima dua buah `Domino` lalu menjumlahkan nilai keduanya menggunakan `nilaiKartu`. Fungsi mengembalikan `true` jika totalnya tepat 12, dan `false` jika tidak.

main membuat dan mengocok satu set domino, lalu menampilkan 5 kartu pertama. Kemudian mengambil satu kartu sebagai target dan memanggil `galiKartu` untuk mencari kartu berikutnya yang suitnya cocok dengan target, menampilkan hasilnya beserta sisa kartu. Selanjutnya program menguji `sepasangKartu` dengan dua pasang kartu berbeda — pertama kartu [5|2] dan [3|2] yang totalnya 12 (sepasang), lalu kartu [6|6] dan [0|0] yang totalnya 12 juga (sepasang) — untuk memverifikasi fungsi bekerja dengan benar.

Tugas 18.3.3

```
package main

import (
    "fmt"
    "math/rand"
)

type Domino struct {
    suit  [2]int
    nilai int
    isBalak bool
}

type Dominoes struct {
    kartu    [28]Domino
    jumlahTersisa int
}

func buatDomino(s1, s2 int) Domino {
    return Domino{
        suit:  [2]int{s1, s2},
        nilai: s1 + s2,
        isBalak: s1 == s2,
    }
}

func buatDominoes() Dominoes {
    var d Dominoes
    idx := 0
    for i := 0; i <= 6; i++ {
        for j := i; j <= 6; j++ {
            d.kartu[idx] = buatDomino(i, j)
            idx++
        }
    }
    d.jumlahTersisa = 28
    return d
}

func kocokKartu(d *Dominoes) {
    for i := 27; i > 0; i-- {
        j := rand.Intn(i + 1)
        d.kartu[i], d.kartu[j] = d.kartu[j], d.kartu[i]
    }
}

func ambilKartu(d *Dominoes) Domino {
    if d.jumlahTersisa == 0 {
        fmt.Println("Tidak ada kartu tersisa!")
    }
}
```

```

        return Domino{}
    }
    kartu := d.kartu[d.jumlahTersisa-1]
    d.jumlahTersisa--
    return kartu
}

func gambarKartu(d Domino, suit int) int {
    if suit == 0 {
        return d.suit[0]
    }
    return d.suit[1]
}

func nilaiKartu(d Domino) int {
    return d.nilai
}

func cetakKartu(d Domino) {
    balak := "bukan balak"
    if d.isBalak {
        balak = "balak"
    }
    fmt.Printf("[%d|%d] nilai=%d (%s)\n", d.suit[0], d.suit[1], d.nilai, balak)
}

func galiKartu(d *Dominoes, target Domino) Domino {
    var hasil Domino
    for d.jumlahTersisa > 0 {
        hasil = ambilKartu(d)
        if hasil.suit[0] == target.suit[0] || hasil.suit[0] == target.suit[1] ||
            hasil.suit[1] == target.suit[0] || hasil.suit[1] == target.suit[1] {
            return hasil
        }
    }
    return hasil
}

func sepasangKartu(a, b Domino) bool {
    return nilaiKartu(a)+nilaiKartu(b) == 12
}

func main() {
    set := buatDominoes()
    kocokKartu(&set)

    fmt.Println("=== Set Domino Setelah Dikocok (5 kartu pertama) ===")
    for i := 0; i < 5; i++ {
        cetakKartu(set.kartu[set.jumlahTersisa-1-i])
    }
}

```

```

    fmt.Println("\n=== galiKartu ===")
    target := ambilKartu(&set)
    fmt.Print("Kartu target: ")
    cetakKartu(target)
    fmt.Printf("Jumlah kartu tersisa setelah ambil target: %d\n", set.jumlahTersisa)

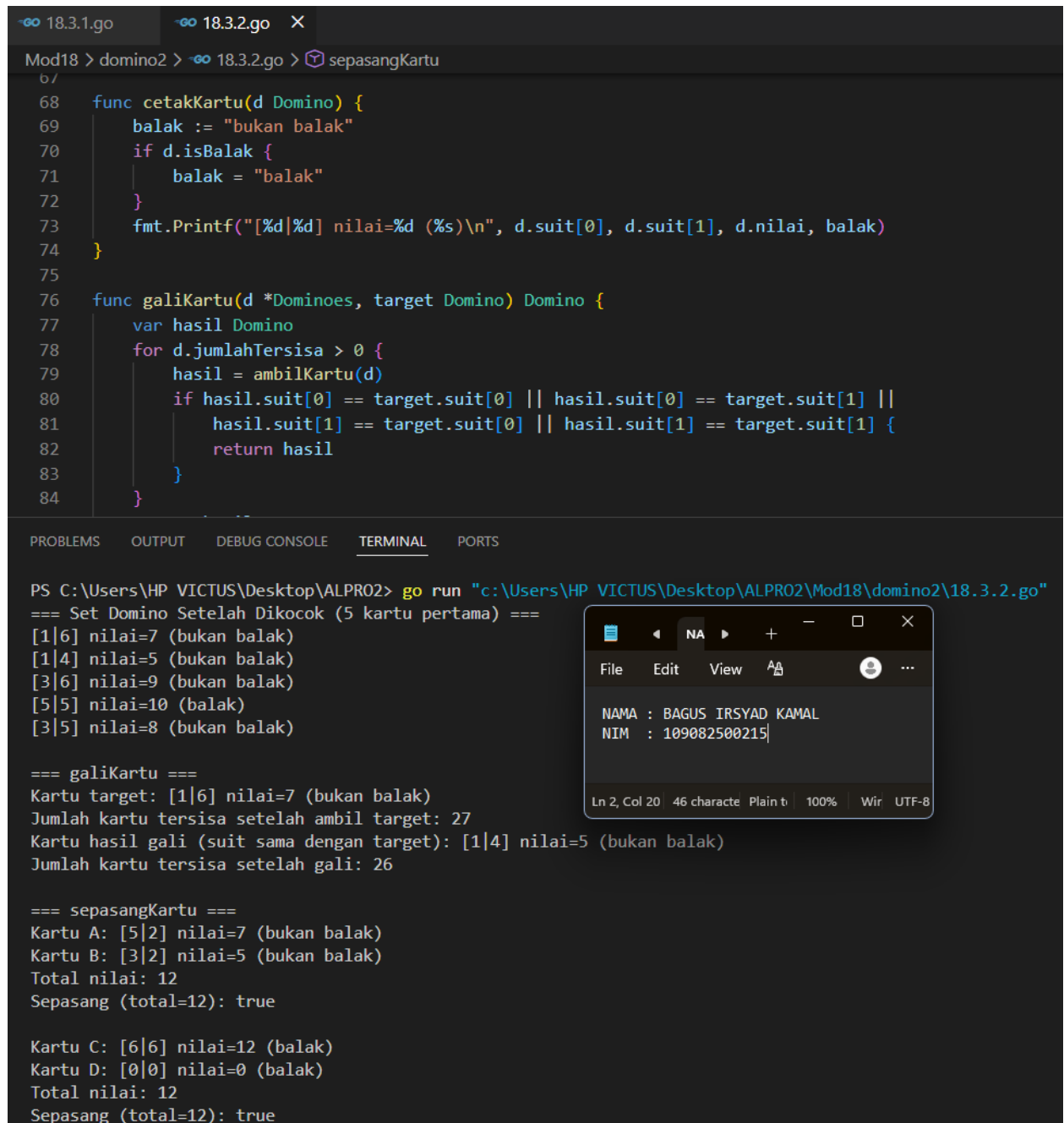
    hasil := galiKartu(&set, target)
    fmt.Print("Kartu hasil gali (suit sama dengan target): ")
    cetakKartu(hasil)
    fmt.Printf("Jumlah kartu tersisa setelah gali: %d\n", set.jumlahTersisa)

    fmt.Println("\n=== sepasangKartu ===")
    kartuA := buatDomino(5, 2)
    kartuB := buatDomino(3, 2)
    fmt.Print("Kartu A: ")
    cetakKartu(kartuA)
    fmt.Print("Kartu B: ")
    cetakKartu(kartuB)
    fmt.Printf("Total nilai: %d\n", nilaiKartu(kartuA)+nilaiKartu(kartuB))
    fmt.Printf("Sepasang (total=12): %v\n", sepasangKartu(kartuA, kartuB))

    kartuC := buatDomino(6, 6)
    kartuD := buatDomino(0, 0)
    fmt.Print("\nKartu C: ")
    cetakKartu(kartuC)
    fmt.Print("Kartu D: ")
    cetakKartu(kartuD)
    fmt.Printf("Total nilai: %d\n", nilaiKartu(kartuC)+nilaiKartu(kartuD))
    fmt.Printf("Sepasang (total=12): %v\n", sepasangKartu(kartuC, kartuD))
}

```

Screenshots Output



```
Mod18 > domino2 > 18.3.2.go > sepaasangKartu
67
68 func cetakKartu(d Domino) {
69     balak := "bukan balak"
70     if d.isBalak {
71         balak = "balak"
72     }
73     fmt.Printf("[%d|%d] nilai=%d (%s)\n", d.suit[0], d.suit[1], d.nilai, balak)
74 }
75
76 func galiKartu(d *Dominoes, target Domino) Domino {
77     var hasil Domino
78     for d.jumlahTersisa > 0 {
79         hasil = ambilKartu(d)
80         if hasil.suit[0] == target.suit[0] || hasil.suit[0] == target.suit[1] ||
81             hasil.suit[1] == target.suit[0] || hasil.suit[1] == target.suit[1] {
82             return hasil
83         }
84     }
85 }
```

PS C:\Users\HP VICTUS\Desktop\ALPRO02> go run "c:\Users\HP VICTUS\Desktop\ALPRO02\Mod18\domino2\18.3.2.go"

=== Set Domino Setelah Dikocok (5 kartu pertama) ===

[1|6] nilai=7 (bukan balak)
[1|4] nilai=5 (bukan balak)
[3|6] nilai=9 (bukan balak)
[5|5] nilai=10 (balak)
[3|5] nilai=8 (bukan balak)

=== galiKartu ===

Kartu target: [1|6] nilai=7 (bukan balak)
Jumlah kartu tersisa setelah ambil target: 27
Kartu hasil gali (suit sama dengan target): [1|4] nilai=5 (bukan balak)
Jumlah kartu tersisa setelah gali: 26

=== sepaasangKartu ===

Kartu A: [5|2] nilai=7 (bukan balak)
Kartu B: [3|2] nilai=5 (bukan balak)
Total nilai: 12
Sepasang (total=12): true

Kartu C: [6|6] nilai=12 (balak)
Kartu D: [0|0] nilai=0 (balak)
Total nilai: 12
Sepasang (total=12): true

Deskripsi:

Program ini merupakan pengembangan dari soal 18.3.2. Tujuannya mengimplementasikan permainan domino Gapleh, yaitu membuat rangkaian kartu domino yang sambung-menyambung berdasarkan kesamaan gambar (suit). Setiap kartu yang ditambahkan ke rangkaian harus memiliki setidaknya satu sisi yang nilainya sama dengan sisi ujung rangkaian yang sudah terbentuk.

Fungsi-fungsi yang digunakan:

`galiKartu` menjadi inti dari permainan Gapleh. Fungsi ini menerima pointer `Dominoes` dan sebuah `Domino` sebagai kartu ujung rangkaian saat ini, lalu mengambil kartu dari tumpukan satu per satu menggunakan `ambilKartu` hingga ditemukan kartu yang memiliki sisi (suit) dengan nilai sama dengan salah satu sisi kartu ujung. Kartu yang cocok tersebut kemudian menjadi ujung rangkaian yang baru dan siap disambung dengan kartu berikutnya. Proses ini terus berulang selama masih ada kartu di tumpukan, membentuk rantai kartu yang saling terhubung berdasarkan kesamaan suit.

`ambilKartu` berperan sebagai operasi dasar yang digunakan `galiKartu` untuk mengambil kartu dari tumpukan satu per satu selama proses pencarian kartu yang cocok berlangsung.

`gambarKartu` dan `nilaiKartu` digunakan untuk membaca informasi sisi dan nilai kartu sehingga perbandingan suit antar kartu dalam rangkaian dapat dilakukan dengan benar.

main mensimulasikan permainan Gapleh dengan mengocok set domino terlebih dahulu, lalu mengambil satu kartu sebagai kartu awal rangkaian. Dari kartu awal tersebut, program memanggil `galiKartu` berulang untuk mencari kartu berikutnya yang suitnya cocok, sehingga terbentuk rangkaian kartu yang sambung-menyambung. Setiap kartu yang berhasil disambung ditampilkan beserta sisa kartu di tumpukan, hingga tumpukan habis atau rangkaian tidak dapat dilanjutkan lagi.

Tugas 18.3.4

```
package main

import "fmt"

var (
    input []rune
    pos  int
)

func start(s string) {
    input = []rune(s)
    pos = 0
}

func maju() {
    if pos < len(input) {
        pos++
    }
}

func eop() bool {
    return pos >= len(input) || input[pos] == '.'
}

func cc() rune {
    return input[pos]
}

func bacaSemua() {
    fmt.Print("Karakter terbaca: ")
    for !eop() {
        fmt.Printf("%c", cc())
        maju()
    }
    fmt.Println()
}

func hitungKarakter() int {
    jumlah := 0
    for !eop() {
        jumlah++
        maju()
    }
    return jumlah
}

func hitungHurufA() int {
    jumlah := 0
    for !eop() {
```

```

        if cc() == 'A' {
            jumlah++
        }
        maju()
    }
    return jumlah
}

func hitungFrekuensiA() float64 {
    jumlahA := 0
    total := 0
    for !eop() {
        total++
        if cc() == 'A' {
            jumlahA++
        }
        maju()
    }
    if total == 0 {
        return 0
    }
    return float64(jumlahA) / float64(total)
}

func hitungKataLE() int {
    jumlah := 0
    prev := rune(0)
    for !eop() {
        if prev == 'L' && cc() == 'E' {
            jumlah++
        }
        prev = cc()
        maju()
    }
    return jumlah
}

func main() {
    teks := "HALO SELAMAT DATANG KE TELKOM UNIVERSITY."

    fmt.Println("Input:", teks)
    fmt.Println()

    start(teks)
    bacaSemua()

    start(teks)
    fmt.Printf("Jumlah karakter: %d\n", hitungKarakter())

    start(teks)

```

```
    fmt.Printf("Jumlah huruf A: %d\n", hitungHurufA())

    start(teks)
    fmt.Printf("Frekuensi huruf A: %.4f\n", hitungFrekuensiA())

    start(teks)
    fmt.Printf("Jumlah kata LE: %d\n", hitungKataLE())
}
```

Screenshots Output

The screenshot displays a Go IDE with three tabs: 18.3.1.go, 18.3.2.go, and 18.3.4.go. The active tab, 18.3.4.go, contains the following Go code:

```
Mod18 > karakter > 18.3.4.go > main
38 func hitungKarakter() int {
45 }
46
47 func hitungHurufA() int {
48     jumlah := 0
49     for !eop() {
50         if cc() == 'A' {
51             jumlah++
52         }
53         maju()
54     }
55     return jumlah
56 }
57
58 func hitungFrekuensiA() float64 {
59     jumlahA := 0
60     total := 0
61     for !eop() {
62         total++
63         if cc() == 'A' {
64             jumlahA++
65         }
66         maju()
67     }
68     if total == 0 {
69         return 0
70     }
71     return float64(jumlahA) / float64(total)
72 }
73
74 func hitungKataLE() int {
75     jumlah := 0
```

An output window is open, displaying the following text:

```
NA
File Edit View
NAMA : BAGUS IRSYAD KAMAL
NIM : 109082500215
Ln 2, Col 20 46 character Plain t 100% Wir UTF-8
```

The terminal window at the bottom shows the execution of the program:

```
PS C:\Users\HP VICTUS\Desktop\ALPRO2> go run "c:\Users\HP VICTUS\Desktop\ALPRO2\Mod18\karakter\18.3.4.go"
Input: HALO SELAMAT DATANG KE TELKOM UNIVERSITY.

Karakter terbaca: HALO SELAMAT DATANG KE TELKOM UNIVERSITY
Jumlah karakter: 40
Jumlah huruf A: 5
Frekuensi huruf A: 0.1250
Jumlah kata LE: 0
PS C:\Users\HP VICTUS\Desktop\ALPRO2>
```

Deskripsi:

Program ini merupakan implementasi soal 18.3.4. Tujuannya membangun mesin abstrak karakter yang bekerja terhadap untaian karakter yang diakhiri penanda titik ("."), lengkap dengan sejumlah operasi dasar dan algoritma-algoritma yang memanfaatkan operasi dasar tersebut.

Variabel global yang digunakan adalah input bertipe slice rune yang menyimpan seluruh karakter untaian, dan pos bertipe int yang menyimpan posisi pembacaan mesin saat ini.

Operasi dasar mesin karakter:

start menerima string lalu mengubahnya menjadi slice rune dan mereset posisi pembacaan ke awal, sehingga mesin siap membaca dari karakter pertama. maju memajukan posisi pembacaan satu langkah ke karakter berikutnya selama posisi belum melewati batas untaian. eop mengembalikan true apabila posisi sudah melewati batas untaian atau karakter yang sedang dibaca adalah titik, menandakan akhir dari rangkaian karakter. cc mengembalikan karakter yang sedang berada di posisi pembacaan saat ini tanpa menggeser posisi.

Algoritma-algoritma yang dibangun:

bacaSemua membaca dan mencetak seluruh karakter satu per satu dengan memanggil cc lalu maju berulang hingga eop bernilai true. hitungKarakter menghitung total karakter yang terbaca dengan menambah penghitung setiap kali maju dipanggil hingga mencapai penanda akhir. hitungHurufA menghitung berapa banyak karakter 'A' yang ditemukan dengan membandingkan hasil cc di setiap posisi. hitungFrekuensiA menghitung frekuensi kemunculan huruf 'A' terhadap seluruh karakter dengan membagi jumlah 'A' yang ditemukan dengan total karakter, hasilnya dikembalikan sebagai float64. hitungKataLE menghitung berapa banyak pasangan berturutan huruf 'L' diikuti 'E' dengan menyimpan karakter sebelumnya di variabel prev dan membandingkannya dengan karakter saat ini di setiap langkah.

main menggunakan satu teks contoh dan memanggil start sebelum setiap algoritma agar posisi pembacaan selalu direset ke awal, lalu menampilkan hasil dari setiap operasi secara berurutan.